

PacketArch

Technical Deep-Dive

Protocol-Accurate OT Traffic

Live + PCAP

Cyber Vision Ready

Version 1.5 | May 2026

For security architects, OT network engineers, and SOC leads

Architecture at a Glance

Server (single Docker host)

- **Frontend:** React 18 + Vite + TypeScript, @xyflow/react canvas, Ant Design dark theme, Zustand stores
- **Backend:** FastAPI + SQLAlchemy 2.0 async, Scapy 2.6, Celery workers
- **Data:** PostgreSQL 16 (scenarios, templates, audit), Redis (queue + cache)
- **Edge:** nginx reverse proxy, self-signed or operator-supplied TLS

Distributed plane

- **Traffic Agent:** Docker container, **outbound** WebSocket to server (`wss://host/ws/agent`) — no inbound ports
- **MCP server:** JSON-RPC 2.0 + SSE for AI tool calls
- **AI providers:** Anthropic, OpenAI, Cisco CIRCUIT (pluggable)
- **Cyber Vision:** REST integration for device match + enrich

```
[Browser] --HTTPS--> [nginx] --> [FastAPI] <--> [Postgres / Redis]
                        ^
                        | wss (token-auth)
[Agent A] [Agent B] [Agent N] <-- any reachable network
```

Protocol Engine Pattern

Every protocol is a stateful engine with the same three-method contract. The same engine powers PCAP generation and live injection.

```
class ProtocolEngine(ABC):
    @abstractmethod
    def generate_startup_sequence(self) -> list[PacketEvent]:
        """TCP handshake, vendor handshake, identity queries (SZL, MEI, List Identity)."""

    @abstractmethod
    def generate_poll_cycle(self) -> list[PacketEvent]:
        """One protocol exchange – request + response + retries if applicable."""

    @abstractmethod
    def generate_shutdown_sequence(self) -> list[PacketEvent]:
        """Graceful close: FIN/RST, BACnet IAmGone, EtherNet/IP UnRegisterSession."""
```

- State machines via `python-statemachine` (Modbus, EtherNet/IP, PROFINET each have distinct flows)
- Engines emit `PacketEvent(timestamp_ms, flow_id, packet_bytes)` — opaque to the dispatcher
- New protocol = new subclass + register in `PROTOCOL_TO_IDENTITY_KEY`

Supported Protocols

Protocol	Port(s)	Engine status	Notes
Modbus TCP	502	Production	MEI discovery in startup; FC1/2/3/4/5/6/15/16/22/23
EtherNet/IP / CIP	44818 TCP, 2222 UDP	Production	24-byte encap header (<HHIIQI), CIP Identity Object, List Identity
PROFINET	Layer 2 + DCP	Production	Cyclic real-time, alarm, DCP Identify Response, station_name
S7comm	102 TCP	Production	COTP + S7 setup + SZL query sequence in startup
BACnet/IP	47808 UDP	Production	I-Am, ReadProperty, COV, firmware_revision merge
SNMP / NTCIP	161, 162 UDP	Production	Universal discovery guardrail across all fingerprinted devices

Variants (`profisafe` , `s7comm_plus` , `cip_safety` , `enip` , `bacnet_ip`) alias to parent engines via `PROTOCOL_ALIASES` in the agent so safety-flavored flows are not silently dropped.

The Unified Traffic Engine

One orchestrator, two outputs. Same engines, same identities, same timing model — only the sink differs.

Timed mode → PCAP

- `duration_ms = 60_000`
- Sink: `PcapOutput`
- Scapy `PcapWriter` with explicit `write_header(pkt)` then `write_packet(bytes, sec=...)`
- Output: portable `.pcap` for analysis, replay, sensor tuning

Perpetual mode → Live

- `duration_ms = None`
- Sink: `LiveOutput` (Scapy `sendp` on agent interface)
- Runs until `STOP_SCENARIO`
- Wall-clock alignment per virtual-time event

```
# Same call, different sink
UnifiedOrchestrator(output=PcapOutput("out.pcap"), duration_ms=60_000)
UnifiedOrchestrator(output=LiveOutput(iface="ens3"), duration_ms=None)
```

Agent receives engines via **Docker build staging**: `backend/app/protocol_engines/` is copied into `docker/packetarch-agent/_shared/` at image build, eliminating engine drift between PCAP and live paths.

Device Template Catalog

295 templates across 18 vendor modules. One package, one source of truth, zero duplication between PCAP and agent.

Layout

`backend/app/services/device_templates/`

- `_types.py` — `DeviceTemplate`, `TemplateSource`
- `_registry.py` — central register
- `_api.py` — query helpers
- `_fingerprints.py` — firmware/identity merge
- `vendors/siemens.py` (52), `rockwell.py` (38), `schneider.py` (38), `honeywell.py` (17), `abb.py` (13), `yokogawa.py` (13), `cisco.py` (11), `emerson.py` (9), `ge.py` (9), `sel.py` (6), `hms.py` (5), plus 7 industry-grouped files

Each template carries

- Vendor + model + firmware
- IEEE-verified OUI prefix (e.g. Siemens `00:0E:8C`)
- CIP / SNMP / PROFINET / S7 / BACnet / Modbus identity blocks
- Default response timing distribution
- Behavioral hints (poll cadence, write tolerance)

Sources

- `VENDOR_BUILTIN` — shipped catalog
- `USER_CREATED` — operator-defined

Fingerprint Application

The `FingerprintApplicator` is what makes a generated device look real to Cyber Vision and to passive monitors.

```
applicator = FingerprintApplicator(template, instance_seed=device_id)
applicator.get_identity_response("ethernet_ip")    # CIP Identity reply
applicator.get_identity_response("s7comm")        # SZL block 0x0011
applicator.get_identity_response("snmp")          # sysDescr / sysObjectID / sysName
applicator.get_identity_response("bacnet")         # firmware_revision, application_software_version
applicator.get_identity_response("modbus")        # MEI Read Device Identification
```

- **Lazy identity init** (`__getattr__`) — identity dicts built on first access
- **Per-instance serials** — uniqueness within a scenario *and* across scenarios so CV does not merge entities
- **Vendor enterprise OIDs** — 35+ vendors mapped to IANA PEN OIDs in `vendor_oui.py::VENDOR_ENTERPRISE_OIDS` (fixes the classic "everything looks Cisco" bug)
- **Firmware merge** — `firmware.version` propagated into `bacnet_identity.firmware_revision`, `s7_identity.firmware_version`, `snmp_identity.firmware_version`

Verified by `backend/scripts/cv_fingerprint_test.py` — 39 checks across 6 protocols.

The Five Realism Dimensions

Every scenario — template, AI-generated, or imported — is graded against these. Readiness checks block deploy if any fail.

#	Dimension	What it enforces
1	Device Naming	Site-coherent, role-aware, unique. <code>Assembly_Line_PLC_01</code> not <code>device_001</code> .
2	Protocol Accuracy	Vendor fingerprint dictates allowed protocols. Siemens gets S7/PROFINET, not EtherNet/IP. Bidirectional repair.
3	Completeness	Every device participates in at least one flow. Orphans get SNMP monitoring fallback so CV can still fingerprint.
4	Conduit Compliance	Cross-zone flows require an IEC 62443 conduit. Intra-zone traffic is unrestricted.
5	Vendor-Realistic MAC	OUI prefix matches declared vendor (IEEE-verified). MAC regen follows fingerprint changes.

Surfaced via the **Readiness panel**, the **AI Review** scoring rubric, and one-click remediation actions.

IEC 62443 Conduit Compliance

Zones get drawn on the canvas. Conduits are the only legal path between them.

Mechanism

- Each `Zone` declares Purdue level (0-5) and security level
- `Conduit` is a typed edge between two zones with an allow-list of protocols
- Auto-generated when omitted (Purdue-aware defaults)
- Compliance service checks every flow at readiness time and at runtime

Cell isolation mode

- Scenario-level flag (`cell_isolation: strict | relaxed`)
- Strict: drops any inter-cell flow without an explicit conduit
- Relaxed: warns, still ships
- Badged in Studio, deployment cards, and the live dashboard

```
# Conduit definition (auto-generated for typical Purdue boundaries)
Conduit(
    src_zone="L3_Operations", dst_zone="L2_Supervisory",
    allowed_protocols=["modbus_tcp", "ethernet_ip", "opc_ua"],
    direction="bidirectional", security_level=2,
)
```

Adaptive Traffic Generation

Real OT networks are not metronomes. The `AdaptiveController` is a composition peer on `UnifiedOrchestrator` that mutates each next-poll interval before it fires.

Composition order

1. **Phase** — startup/steady/maintenance/shutdown rate multiplier
2. **Schedule** — time-of-day macro (`industrial_24h`, `office_hours`, `data_center`, `constant`)
3. **Directives** — operator overrides via `ADAPT_TRAFFIC`
4. **Micro** — bounded random walk ($\pm 5\%$), 0.2% retransmits, 1–2 h connection resets

Vendor personality

`VENDOR_TRAITS` for 15 vendors: consistency, warmup, eagerness — derived using `stdlib` `random` only, no `scipy` dependency.

Atomic swap

`apply_directives()` builds a replacement controller off-thread, then swaps in. The orchestrator never observes a half-applied config.

50 ms floor on any computed interval. 73 tests (64 unit + 9 integration) gate the controller.

Broadcast / Multicast Ambient Layer

A scenario is more than its named flows. The `BackgroundNoiseGenerator` produces the ambient L2/L3 chatter monitors expect to see.

Builders

- **ARP** — gratuitous + request/reply
- **NTP** — client polls with vendor-correct stratum
- **LLDP** — port descr, system name, capability TLV
- **STP/RSTP** — BPDU to `01:80:C2:00:00:00`
- **CDP** — Cisco-only to `01:00:0C:CC:CC:CC`
- **DHCP** — DORA per device

Per-device gating via `_should_lldp/stp/dhcp/cdp/igmp/...()` so a Siemens HMI does not emit CDP and a Schneider PLC does not emit Cisco-specific multicast.

Zone-aware builders

- **IGMPv2** — group reports per device subscription
- **BACnet** — I-Am, Who-Is broadcasts collected from `_zone_devices`
- **PROFINET DCP** — Identify Response per station
- **SNMP traps** — coldStart, linkUp, vendor-enterprise

Process Simulation Engine

Sensor values drift, react, and fault — they are not random numbers. The process-sim controller injects values into `PayloadGenerator.states` so every poll response reflects a coherent process model.

Math

- **First-order lag** with optional Gaussian noise on every variable
- **Forward-Euler ODE** integration
- **Setpoint state machine** transitioning with phase (`STARTUP` → `WARMING_UP` → `STEADY` → `MAINTENANCE`)

Faults

- `FaultScenario` with causal chains (`pump_seizure` → `flow_drop` → `tank_overflow`)
- Per-effect `delay_ms` for realistic propagation
- Operator-triggered or scheduled

```
# Templates per vertical
manufacturing    -> CNC machining cell (spindle load, coolant flow)
water            -> treatment train (pH, chlorine, turbidity, valve)
building_automation-> HVAC (zone temp, damper position, supply air)
oil_gas          -> wellhead (casing pressure, choke position)
```

Auto-enabled when the scenario's vertical has a template — PCAP and live behave identically.

Live Attack Simulation

`AttackOrchestrator` is the third composition peer (after adaptive + process-sim). It schedules `attack_stage_tick` events on the same virtual-time heap, then bridges Scapy packets back to `PacketEvent`.

17+ action generators

- Recon: `port_scan`, `snmp_walk`, `modbus_unit_scan`
- C2: `c2_beacon`, dns tunneling
- ICS write: `modbus_write_register`, `enip_forward_open`, `s7_write_db`
- Disruption: `enip_reset`, `s7_stop_cpu`
- Registered via `@register_action("name")` decorator

Six playbooks (as code, not DB rows)

Playbook	Inspired by
<code>TRITON_LIKE</code>	Safety controller attack
<code>PIPEDREAM_LIKE</code>	Multi-protocol toolkit
<code>INDUSTROYER_LIKE</code>	Grid disruption
<code>HAVEX_LIKE</code>	OPC reconnaissance
<code>INSIDER_THREAT</code>	Authorized misuse
<code>NETWORK_RECON</code>	Pre-attack mapping

Live UI: `AttackPanel` (4-state machine), `KillChainTimeline` overlayed on `DeploymentCard`, runtime control via `/api/v1/attacks/{id}/start|stop|advance|pause`.

Attack After-Action Report

Every fired action is captured in-flight; the report rides the existing agent → backend STATUS pipeline (no new WebSocket frame).

```
@dataclass
class ActionReport:
    action_type: str
    fire_count: int
    packets_emitted: int
    targets: list[str]          # device IDs hit
    iocs: dict[str, Any]        # attacker IP, target IPs/ports, register addrs,
                                # function codes, SNMP communities, beacon patterns
    started_at_ms: int
    completed_at_ms: int | None
```

- Per-stage `planned_duration` vs `actual_duration` and status
- **MITRE ATT&CK for ICS** technique mapping, planned-mode + actual-mode panels (fired techniques highlighted)
- Persisted into `scenario.definition.attack_history[]` on `completed` | `stopped` (cap 50, dedup by `playbook_id` + `started_at`)
- `GET /attacks/{id}/report` (live or history) + `GET /attacks/{id}/history`
- JSON download per report for hand-off to detection engineering

Cisco Cyber Vision Integration

PacketArch is built to *survive* CV's classification heuristics, not just send packets at it.

Match flows

- **MAC** match — 100% confidence (vendor OUI exact)
- **IP** match — 95% confidence (within scenario `/16`)
- Bidirectional compare panel: PacketArch device ↔ CV component
- One-click enrich: push vendor / model / firmware / serial back to CV

Anti-merge guarantees

- Per-role unique `sys_object_id`, `model_name`, `sys_descr` so distinct PacketArch roles don't collapse into one CV entity
- Site identity rail makes device labels globally unique across scenarios
- OUI rotation differentiates at L2 even within same-shop profile

Configured at **Settings** → **Cyber Vision** (URL + API token). Live: `services/cyber_vision_service.py`, `routes/cyber_vision.py`, `pages/CyberVisionPage.tsx`.

Traffic Agent

Docker container, phone-home model, no inbound ports — installable through corporate NAT and most lab firewalls.

Wire protocol

Server → Agent

START_SCENARIO , STOP_SCENARIO , UPDATE_SCENARIO ,
ADAPT_TRAFFIC , START_ATTACK , STOP_ATTACK , ADVANCE_STAGE ,
LIST_INTERFACES , UPDATE_AGENT , PING

Agent → Server

STATUS , INTERFACES , ERROR , HEARTBEAT (CPU / mem /
version), UPDATE_STATUS

Centralized updates

1. **Build Image** in Settings → Agents (build + save tarball)
2. Open agent → **Update** (sends UPDATE_AGENT)
3. Agent downloads tarball, docker load , restarts

Requires Docker socket mount + agent online.

Cloud-link daemon

Heartbeats for eWON / bastion / jump-server run on a **wall-clock thread**, not the orchestrator heap — prevents PROFINET-cyclic flows from starving external comms.

Deployment Topologies

Single host (dev)

- `docker compose -f docker-compose.dev.yml up -d`
- `poetry run uvicorn` + `pnpm dev`
- Backend `:8001`, Frontend `:3001`,
Postgres `:5432`, Redis `:6379`

Single host (prod)

- Same machine as dev
- `docker compose up -d --build`
- nginx `:443` with self-signed or operator cert
- First-run setup wizard claims admin

Multi-lab offline bundle

- `scripts/build-release.sh` produces `packetarch-<ver>-offline.tar.gz`
- `docker save` 'd images + `compose` + `install.sh` + docs
- `install.sh` generates `.env` with fresh secrets
- Backup / restore scripts ship in-bundle

PCAP-only variant

`PCAP_ONLY=1 scripts/build-release.sh` → `LIVE_TRAFFIC_ENABLED=false`. Agent install bundle not shipped, `/ws/agent` not mounted, live routes return 503. Attack playbooks + adaptive timing still work **per PCAP** via `GenerationRequest.attack_config` / `adaptive_config`.

Scenario Authoring

Visual canvas

- `@xyflow/react` with custom `DeviceNode`, resizable `ZoneNode`, color-coded `FlowEdge`
- `@dnd-kit` for palette → canvas drag
- Auto-assigned `/16` per scenario (`10.{n}.0.0/16`, hosts from `.10`)
- Undo/redo, Ctrl+S explicit version snapshot

Portable Scenario v1 — import from any LLM or program

- Public JSON Schema at `schemas/packetarch-scenario.v1.json`
- **Capability mode:** write `{type: plc, protocols: [modbus_tcp]}` — importer resolves vendor + model from local catalog (deterministic by `scenario_name` + `device_index`)
- Vendor preference strategies: `preferred`, `diverse`, `any`
- Endpoints: `GET /scenarios/schema/portable.json`, `POST /scenarios/validate/portable`, `POST /scenarios/import/portable`

AI generation

- "Create a water plant with two PLCs and an HMI on the supervisory cell"
- Backed by the `packetarch-scenario-authoring` skill
- Output runs through the same readiness + repair pipeline as templates and imports

AI Architecture

One provider per install, **task-routed model selection** — operators never have to pick a model per feature.

Providers

- `AnthropicProvider` (Opus 4.7 / Sonnet 4.6 / Haiku 4.5)
- `OpenAIProvider` (gpt-5, gpt-5-mini)
- `CircuitProvider` (Cisco internal gateway — gpt-5-nano cleared)
- All implement `BaseAIProvider.chat(messages, skills=[...])`

Routing

```
TASK_MODEL_MAP[provider][AITask] in  
mcp_server/ai_providers/model_router.py
```

Task	Anthropic
CHAT / SCENARIO_GENERATION / REVIEW	Opus 4.7
DESCRIPTION / AI_HELP	Sonnet 4.6
DEVICE_NAMING / SITE_IDENTITY	Haiku 4.5

MCP tool server + Claude Agent Skills

5 skills under `backend/app/ai_services/skills/`: `scenario-authoring`, `fingerprint-validator`, `ics-attack-playbooks`, `device-naming`, `scenario-review`. Loaded once per process, attached per call via `skills=["..."]`, emitted as cacheable system blocks (Anthropic) or inlined (OpenAI).

Site Identity & Naming Rail

Cross-scenario name collisions (`DMZ_Patch_Server` appearing in N scenarios) used to make CV merge devices into one entity. Fixed structurally.

```
# Every template-materialized scenario runs through this
apply_site_naming_pipeline(scenario, vertical, template_id)
# Picks a per-scenario SiteIdentity:
#   plant / operator / city / role-naming patterns
# Then renames every device using architectural_role + zone counters
```

- LLM-picked when AI is enabled (uses `packetarch-device-naming` skill v2.0.0, Haiku 4.5)
- Deterministic-bank fallback otherwise (`_VERTICAL_SITE_BANK` filtered by template name)
- Admin recovery: `POST /admin/scenarios/{id}/regenerate-names` or install-wide variant
- Snapshots a scenario version before mutating

Devices end up with globally-unique labels by construction — no CV merge possible.

Scenario Versioning

Time-machine for `scenario.definition`. Auto-coalesces during editing, explicit on Ctrl+S, rollback creates a safety snapshot.

Storage

- `ScenarioVersion` table — full definition snapshots
- Auto-version every 5+ minutes during edit
- Cap 50 versions per scenario, auto-prune oldest
- Cascade delete with parent scenario

Diff

- `services/scenario_diff.py` — deep field-level
- Ignores canvas position-only changes
- Surfaces add / remove / modify per device, flow, zone

Routes

```
GET    /scenarios/{id}/versions
POST   /scenarios/{id}/versions
GET    /scenarios/{id}/versions/{vid}
PATCH /scenarios/{id}/versions/{vid}
DELETE /scenarios/{id}/versions/{vid}
POST   /scenarios/{id}/versions/{vid}/diff
POST   /scenarios/{id}/versions/{vid}/rollback
```

Rollback snapshots current state first; does **not** touch the IP range allocation.

Feature Flags

Operator-visible gating for AI features and the live-agent half of the platform.

Flag	Default	When off
AI_ENABLED	true	/api/v1/ai/* and /api/v1/mcp/* → 503. UI hides AI tab, AI Create, Generate Description, Explain with AI.
LIVE_TRAFFIC_ENABLED	true	/agents , /deployments , /adaptation , runtime half of /attacks → 503. /ws/agent not mounted. Read endpoints stay open so PCAP UI keeps the playbook list. Sidebar omits Deployments + Live Traffic.

Adding a flag

- 1. Settings in config.py + Features in features.py
- 2. Frontend Features in api/about.ts
- 3. RequireXEnabled dep on the router
- 4. Gate UI via useFeatures() / <FeatureGate>

Surfaced to the frontend via GET /api/v1/about.features .

Release Bundles

Self-contained offline tarball for air-gapped lab deploys.

```
# Build
scripts/build-release.sh           # full
PCAP_ONLY=1 scripts/build-release.sh # PCAP-only variant

# Output
dist/packetarch-1.5.0-offline.tar.gz # ~1.5 GB
dist/packetarch-1.5.0-pcap-offline.tar.gz # ~900 MB
```

Bundle contents (scripts/release-bundle/)

- `install.sh` — `docker load` s images, generates `.env` with `openssl rand`, prints admin password once. `--upgrade` preserves volumes, `--force-env` regenerates.
- `docker-compose.offline.yml` — production compose, no internet pulls
- `packetarch-backup.sh` — `pg_dump` + tar of PCAP volumes + manifest
- `packetarch-restore.sh` — destructive, requires typed `RESTORE` confirmation
- `schemas/`, `docs/`, `THIRD_PARTY_LICENSES.md`

CI: `.github/workflows/release.yml` matrix-builds both variants on `v*` tag → draft GitHub Release.

Security Posture

Transport

- TLS 1.2+ on `:443`, self-signed by default
- Operator cert: drop `server.crt` + `server.key` into `./certs/`; entrypoint copies to nginx live path on every boot — hot-swap with `docker compose restart frontend`
- Agent WebSocket over `wss://` with bearer token

Secrets

- `SECRET_KEY`, `POSTGRES_PASSWORD` generated at install time
- Agent tokens bcrypt-hashed at rest
- AI API keys + CV tokens Fernet-encrypted in `system_settings`
- `.env` excluded from backups by default (`--include-secrets` to opt in)

Data minimization

- **No live PCAP retention** on server (live mode streams to `LiveOutput`, not disk)
- Generated PCAPs land in a separate volume, operator-managed retention
- Audit log for AI calls (`ai_call_audit`) tracks token + cost

Licensing

- **GPL-3.0** (driven by Scapy GPLv2 dependency)
- Owner strings in `backend/app/core/version.py` — never duplicated
- `scripts/add_copyright_headers.py` enforced via pre-commit
- `THIRD_PARTY_LICENSES.md` regenerated from poetry + pnpm trees

Observability

Agent health

- `HEARTBEAT` every 30 s — CPU, mem, agent version, current scenarios, packet counts per protocol
- `protocol_breakdown` per deployment in `/api/v1/dashboard/live`
- Agent version banner warns when newer image is available

Scenario readiness

- 5-realism scoring per scenario with red / yellow / green
- One-click remediation actions (repair protocols, regenerate MACs, add SNMP fallback flow)
- AI Scenario Review (using `scenario-review` skill) scores categories + emits remediation directives

AI cost dashboard

- `ai_call_audit` table records every provider call (task, model, tokens, latency, status)
- Admin UI: Settings → AI Costs
- CIRCUIT priced `$0` by design (track tokens only)
- Monthly token-quota limits planned

Attack telemetry

- Live kill-chain timeline on `DeploymentCard`
- After-action report with MITRE coverage and IOCs
- History persisted per scenario for retrospective review

Common Workflows

End-to-end sequences operators run most often. Every step below is a single UI click or one `curl` call against the documented API.

1. Sensor / DPI validation

1. Import vertical template (or AI-generate)
2. Deploy live agent on monitored segment
3. Open Cyber Vision Compare → confirm MAC/IP match
4. One-click enrich vendor / model / firmware back into CV
5. Snapshot scenario version for re-runs

2. SOC kill-chain exercise

1. Pick playbook (`TRITON_LIKE` , `INDUSTROYER_LIKE` , ...)
2. Deploy, advance through stages live
3. SOC works the alerts
4. Pull after-action report — MITRE coverage, IOCs, packet counts
5. Diff against detection-engineering ground truth

3. Detection engineering loop

1. Generate PCAP from same scenario + `attack_config`
2. Run through Snort / Suricata / Zeek rule set
3. Inspect after-action JSON for missed techniques
4. Tune rules, re-generate PCAP, repeat — fully offline

4. Multi-lab regression

1. Author once (Portable Scenario v1 JSON)
2. `curl POST /scenarios/import/portable` at every site
3. Schedule daily deploy via cron + REST
4. Compare CV inventory snapshots across sites
5. Roll back any drift via scenario versioning

Every workflow above is repeatable, scriptable, and works in PCAP-only mode when `LIVE_TRAFFIC_ENABLED=false` .

What's Next

Protocol roadmap

- **OPC UA** — Q3 2026
- **DNP3** — Q3 2026
- **IEC 60870-5-104** — Q4 2026
- **GOOSE / SV (IEC 61850)** — exploration

Platform

- Multi-agent coordination (one scenario across N sites)
- Scenario scheduling (cron-driven deploy)
- Plugin SDK for third-party protocol engines
- Token-quota enforcement on AI calls

Detection-engineering tie-in

- Pre-shipped Snort / Suricata / Zeek rule packs aligned with attack playbooks
- Splunk / ELK dashboards for kill-chain replay
- Direct push of after-action reports into SIEM cases

Standards

- Portable Scenario v2 — multi-scenario campaigns, scheduled phases
- Public fingerprint registry feed

Try It

Source + bundles

GitHub `github.com/ip-aegis/PacketArch`

Releases draft attached on every `v*` tag — full + PCAP-only

Install `sudo ./install.sh` from extracted tarball

Documentation

- `CLAUDE.md` — developer guide
- `docs/SCENARIO0_SPEC.md` — Portable Scenario v1
- `docs/ADDING_NEW_PROTOCOLS.md`
- `/api/docs` — Swagger UI

Lab guide

Pair PacketArch with a hardened Cyber Vision sensor, point a SIEM at the agent's interface, run `INDUSTROYER_LIKE` against a Schneider-heavy substation template, and tune detections against the after-action report. Repeatable on a single workstation.